

Code Explanation

Unit Test for Data Processing

This is a unit test class written in Python using the unittest framework to test the functionality of the `clean\_and\_transform\_data` function. The test class `TestDataProcessing` contains methods to set up a Spark session, create sample customer and order data, and test the data processing function.

The test class is designed to ensure that the `clean\_and\_transform\_data` function correctly removes null values and duplicates from the customer and order data, and adds a new column `TotalAmount` to the order data.

Overview of the Code

The code is a unit test class that tests the `clean\_and\_transform\_data` function. It creates a Spark session, generates sample customer and order data, and tests the data processing function to ensure it correctly cleans and transforms the data.

Step 1: Setting Up the Test Environment

This step involves creating a Spark session and a temporary directory for storing test data. The `setUpClass` method is a class method that is called once before running all the tests in the class.

```
@classmethod
def setUpClass(cls):
    cls.spark = SparkSession.builder
        .appName("TestDataProcessing")
        .master("local[*]")
        .getOrCreate()
    cls.temp_dir = tempfile.mkdtemp()
```

Step 2: Creating Sample Customer and Order Data

In this step, sample customer and order data are created using Spark DataFrames. The data includes null values and duplicates to test the data processing function.

```
customer_data = [
    ("C001", "John Doe", "john@example.com", "North"),
    ("C002", "Jane Smith", "jane@example.com", "South"),
    ("C003", "Bob Johnson", "bob@example.com", "East"),
    ("C004", "Alice Brown", "alice@example.com", "West"),
    ("C005", None, "invalid@example.com", "North"), # Null value
    ("C001", "John Doe", "john@example.com", "North") # Duplicate
]

customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

cls.customer_df = cls.spark.createDataFrame(customer_data, schema=customer_schema)
```

Step 3: Testing the Data Processing Function

This step involves testing the `clean\_and\_transform\_data` function using the sample customer and order data. The test checks that null values and duplicates are removed, and a new column `TotalAmount` is added to the order data.

```
def test_clean_and_transform_data(self):
    clean_customer_df, clean_order_df = clean_and_transform_data(self.customer_df, self.order_df)

    self.assertEqual(clean_customer_df.count(), 4)
    self.assertEqual(clean_customer_df.filter(clean_customer_df.CustId == "C001").count(), 1)

    self.assertEqual(clean_order_df.count(), 4)
    self.assertEqual(clean_order_df.filter(clean_order_df.OrderId == "0001").count(), 1)

    order_with_laptop = clean_order_df.filter(clean_order_df.ItemName == "Laptop").collect()[0]
    self.assertEqual(order_with_laptop.TotalAmount, 2000.0) # 1000.0 * 2
```

Step 4: Cleaning Up the Test Environment

In this final step, the Spark session is stopped, and the temporary directory is deleted to clean up the test environment.

```
@classmethod
def tearDownClass(cls):
    cls.spark.stop()
    shutil.rmtree(cls.temp_dir)
```